Build future-ready APIs using Domain-Driven Design (DDD)

Who Am I?

Ashadeepa
Debnath

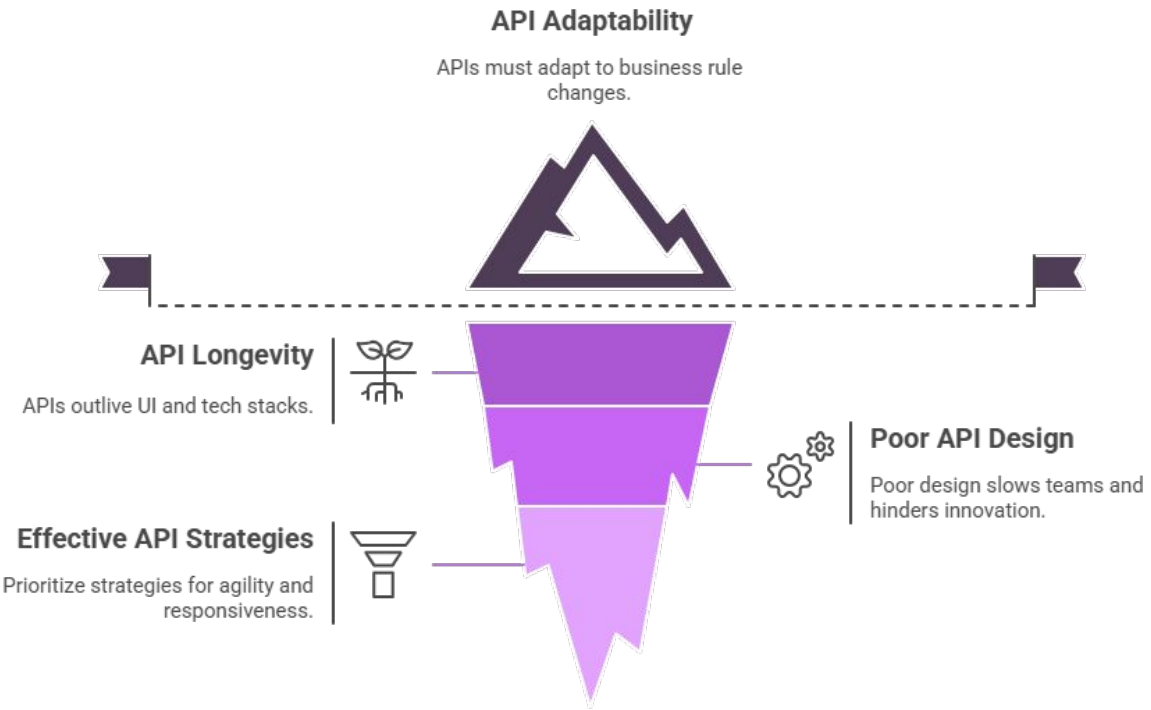
Senior Software Architect

https://github.com/Ashadeepa
https://www.linkedin.com/in/ashadeepa-debnath/
https://in.pinterest.com/ashie1709/ashieverse-creations/my-work/

Ice Breaker

- How many of your APIs are still CRUD-based?
- Have you ever avoided changing an API because it might break consumers?
- Do your APIs reflect business language or database structure?

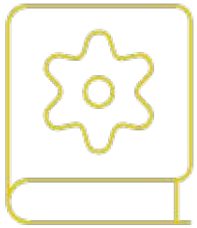
Why Future-Ready APIs Matter



Common API Design Mistakes

- CRUD-driven APIs instead of business-driven APIs
- Leaking database models into API contracts
- One “god service” handling everything
- Versioning chaos and breaking changes

What is Domain-Driven Design (DDD)?



Business Domains

Design software around business domains, not technology.

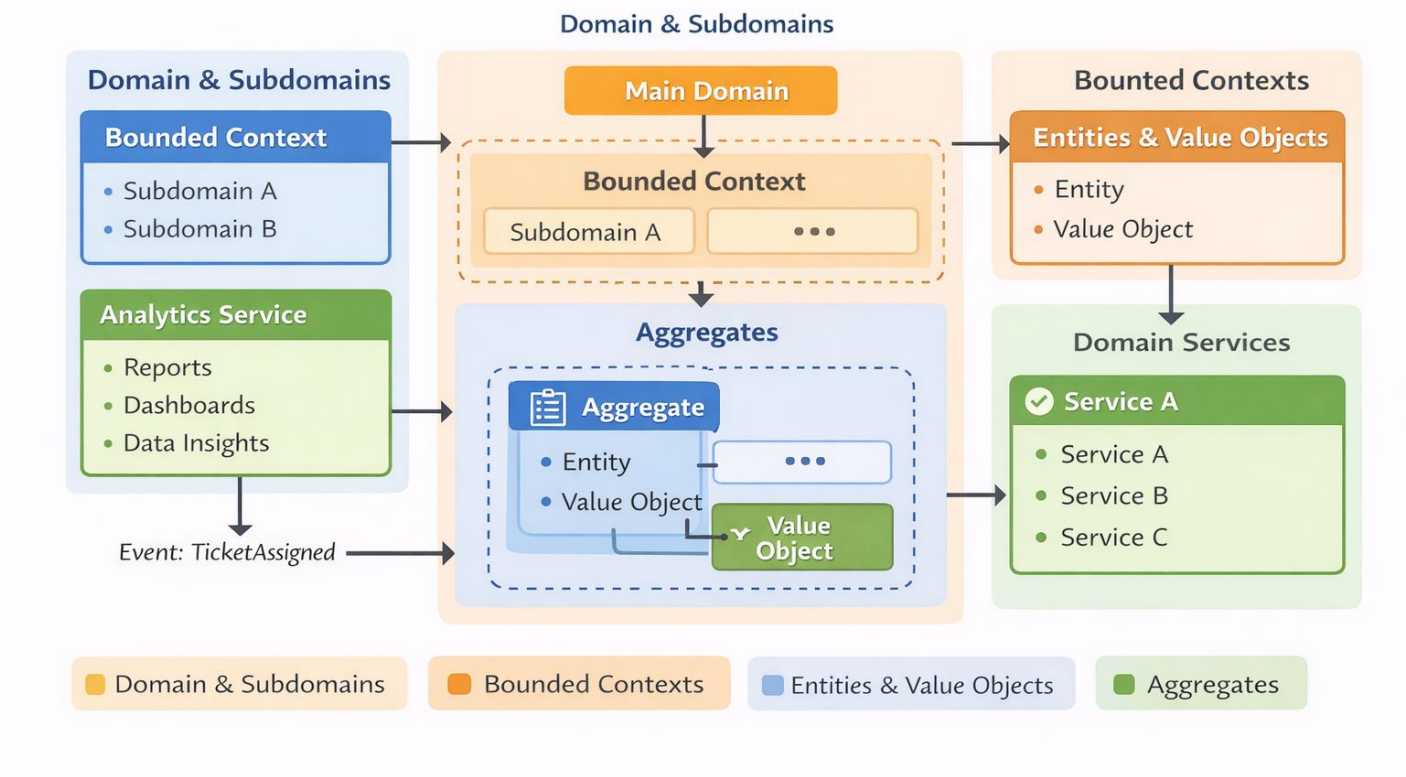
Ubiquitous Language

Shared understanding through Ubiquitous Language.

Business Capabilities

Focus on business capabilities, not endpoints.

Core DDD Building Blocks



Domain

What business problem we're solving

API Mapping:

- Defines the **API landscape** and capability boundaries
- Example: `Orders API`, `Payments API`

Subdomains

Split the problem into business areas

API Mapping:

- Group APIs by business capability
- Example: `Order Management`, `Billing`, `Fulfillment`

Bounded Context

Clear ownership and boundaries

API Mapping:

- One API (or API set) per bounded context
- Independent versioning & deployment
- Example: `POST /orders/place`

Entities & Value Objects

Model the business correctly

API Mapping:

- Entities → Resources with identity
- Value Objects → Request/response objects
- Example:
  - Entity: `/orders/{orderId}`
  - Value Object: `Address`, `Money`

Aggregates

Enforce consistency and rules

API Mapping:

- APIs interact only via **aggregate roots**
- Prevent direct access to internal entities
- Example:
  - `POST /orders/{id}/confirm`

Domain Services

Handle complex business behavior

API Mapping:

- Business-action endpoints (not CRUD)
- Stateless orchestration
- Example:
  - `POST /orders/calculate-price`

Before DDD

```
// Controller directly updates DB
app.put("/updateOrderStatus", (req, res) => {
  db.orders.update({
    id: req.body.id,
    status: req.body.status
  });

  res.send({ success: true });
});
```

After DDD

```
// Domain Entity
class Order {
  constructor(
    public readonly id: string,
    private status: OrderStatus
  ) {}

  confirm() {
    if (this.status !== OrderStatus.CREATED) {
      throw new Error("Invalid state
transition");
    this.status = OrderStatus.CONFIRMED;
  }
}
```

```
// Aggregate Root
class OrderAggregate {
  constructor(private order: Order)
{}
  confirmOrder() {
    this.order.confirm();
  }

  getOrder() {
    return this.order;
  }
}
```

After DDD

```
// API expresses business intent
app.post("/orders/:id/confirm", (req, res) => {
  const order = orderRepository.findById(req.params.id);
  const aggregate = new OrderAggregate(order);

  aggregate.confirmOrder();
  orderRepository.save(aggregate.getOrder());

  res.send({ status: "CONFIRMED" });
});
```

In a nutshell

Aspect	Before	After
API Meaning	Update data	Execute behavior
Business Rules	Missing / scattered	Centralized
State Safety	Fragile	Enforced
Change Risk	High	Low
AI / Workflow Ready	❌	✅

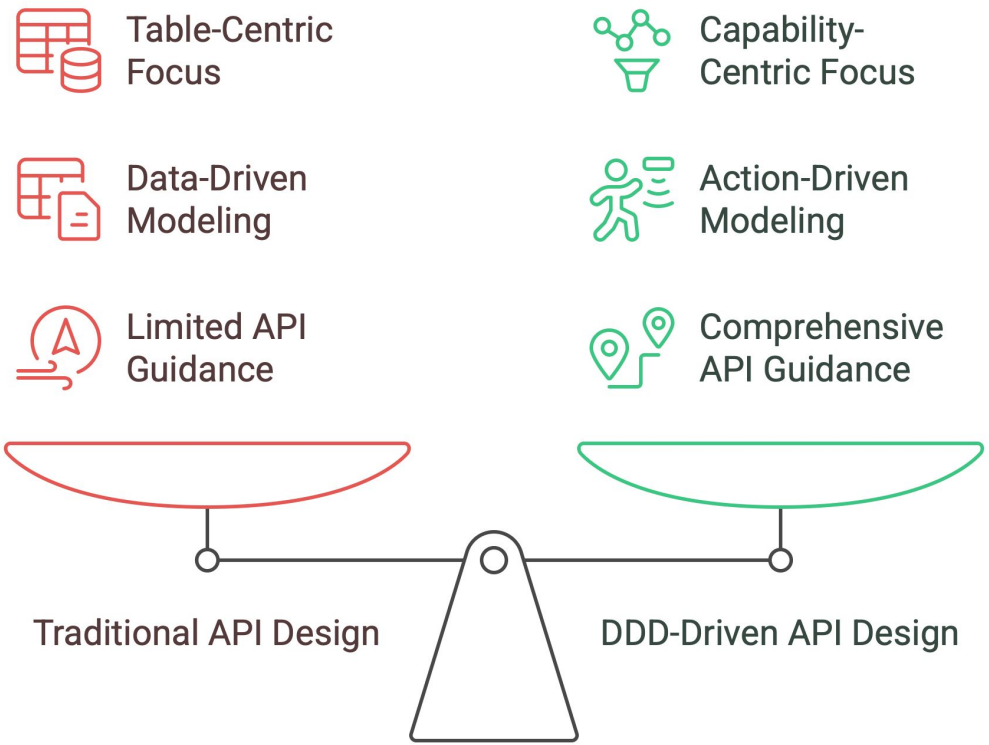
IT'S
DEMO TIME

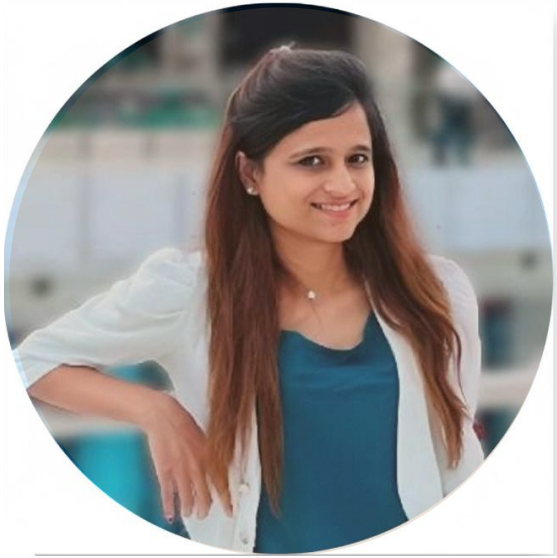
From CRUD APIs to Business Capabilities

- **Stop asking:** *“What tables do I expose?”*
- **Start asking:** *“What business capability does this API represent?”*
- Model APIs around **actions, rules, and outcomes**
- Let DDD guide:
  - *API boundaries*
  - *Endpoint semantics*
  - *Versioning strategy*
  - *Team ownership*

Design APIs the business can grow with — not ones it’s afraid to change.

Shift from Data to Business Capabilities in API Design





Thank You

Name: Ashadeepa Debnath
Designation: Senior Software Architect
LinkedIn Profile:
https://www.linkedin.com/in/ashadeepa-debnath/